

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное
учреждение высшего образования
«Северо-Кавказский федеральный университет»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

КУРСОВАЯ РАБОТА

по дисциплине

«ИНТЕРНЕТ-ТЕХНОЛОГИИ»

на тему:

«Разработка веб-приложения для планирования путешествий»

Выполнил:

Зинченко Сергей Алексеевич

Студент 4 курса группы ПИН-б-з-22

Направления подготовки

09.03.03 Прикладная информатика

заочной формы обучения

(подпись)

Руководитель работы:

(ФИО, должность)

Работа допущена к защите _____
(подпись руководителя) (дата)

Работа выполнена и защищена с оценкой _____

Дата защиты _____

Члены комиссии: _____
(должность) (подпись) (И.О. Фамилия)

Ставрополь, 2026 г.

УТВЕРЖДАЮ
Директор департамента цифровых,
робототехнических систем
и электроники
Азаров И.В.

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники
Направление подготовки 09.03.03 Прикладная информатика
Направленность (профиль) Прикладная информатика в экономике

ЗАДАНИЕ
на курсовую работу
студента Зинченко Сергея Алексеевича
(фамилия, имя, отчество)
по дисциплине Интернет-технологии

1.Тема работы: «Разработка веб-приложения для планирования путешествий»

2.Цель разработать полноценное веб-приложение для планирования путешествий, включающее клиентскую и серверную части, систему аутентификации, инструменты для создания, распределения и отслеживания задач.

3.Технологический стек Python, Django, Django REST Framework, React, TypeScript, Docker.

4.Перечень подлежащих разработке вопросов:

а) по теоретической части Анализ предметной области планирования путешествий, обзор существующих решений, обоснование выбора технологического стека, принципы построения SPA-приложений и REST API.

б) по практической части проектирование моделей данных (Task, User), реализация backend на Django REST Framework с JWT-аутентификацией, разработка frontend на React с TypeScript, реализация Kanban-доски с drag-and-drop, поддержка тёмной и светлой тем, контейнеризация с Docker и развёртывание на VPS.

5.Исходные данные:

техническое задание на разработку веб-приложения для планирования путешествий, документация по Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

6. Список рекомендуемой литературы: официальная документация Django, Django REST Framework, React, TypeScript, Tailwind CSS, Docker.

7. Контрольные сроки представления отдельных разделов курсовой работы:

25 % - _____ “ ____ ” _____ 20_

г.

50 % - _____ “ ____ ” _____ 20_

г.

75 % - _____ “ ____ ” _____ 20_

г.

100 % - _____ “ ____ ” _____ 20_

г.

8. Срок защиты студентом курсовой работы “ ____ ” _____ 20_ г.

Дата выдачи задания “ ____ ” _____ 20_ г.

Руководитель курсовой работы

(ученая степень, звание) (личная подпись) (инициалы, фамилия)

Задание принял(а) к исполнению студент(ка) _____ формы обучения
____ курса ____ группы _____

(личная подпись)

(инициалы, фамилия)

Отзыв

на курсовой проект студента Зинченко Сергея Алексеевича
Тема работы: «Разработка веб-приложения для планирования путешествий».

Курсовая работа посвящена актуальной теме разработки систем планирования путешествий. Продемонстрированы хорошие навыки проектирования и разработки веб-приложений.

В ходе выполнения работы были использованы современные технологии:

- Backend: Django и Django REST Framework.
- Frontend: React и экосистема (Vite, TailwindCSS).
- Контейнеризация: Docker и Docker Compose.

Работа выполнена на высоком техническом уровне. Реализованный функционал соответствует техническому заданию. Приложение имеет продуманную архитектуру (клиент-серверная) и код, соответствующий стандартам разработки.

Положительные стороны:

- Использование разделения ответственности (Frontend/Backend).
- Настройка Docker окружения для быстрого развертывания.
- Реализация JWT авторизации и REST API.
- Поддержка Kanban-доски с drag-and-drop.

Недостатки: Существенных недостатков не выявлено.

Заключение: Студент заслуживает оценки «_____».

Научный руководитель: Старший преподаватель Щеголев А.А.

Дата: _____

Содержание

Отзыв	4
Введение	7
1 Анализ предметной области.....	9
1.1 Постановка задачи	9
1.2 Обзор существующих решений	10
1.3 Обоснование выбора технологии разработки	11
2 Техническое задание.....	14
2.1 Функциональные требования.....	14
2.2 Нефункциональные требования	15
3 Проектирование системы	17
3.1 Архитектура системы	17
3.2 Проектирование базы данных	18
3.3 Проектирование REST API.....	21
3.4 Проектирование пользовательского интерфейса	23
4 Реализация	24
4.1 Технологический стек.....	24
4.2 Реализация поездок и маршрутов	26
4.3 Реализация учёта расходов.....	27
4.4 Реализация списка вещей	27
4.5 Реализация бронирований	27
4.6 Аутентификация на основе JWT	28
4.7 Контейнеризация и развёртывание	29
5 ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА	31
5.1 Страница авторизации	31
5.2 Главная страница	32
5.3 Каталог туров	33
5.4 Форма создания тура	34

5.5 Оформление бронирования	35
6 Тестирование.....	36
6.1 Методика тестирования	36
6.2 Результаты функционального тестирования	36
Заключение.....	40
Список использованных источников.....	42

Введение

Современные информационные технологии активно проникают во все сферы повседневной жизни, существенно меняя привычные процессы. Одной из наиболее динамично развивающихся областей является сфера туризма и путешествий. В условиях роста популярности самостоятельных поездок путешественникам необходимы удобные инструменты для планирования маршрутов, учёта расходов, бронирования услуг и сбора вещей. В связи с этим разработка современных веб-приложений для планирования путешествий является актуальной задачей.

Объектом исследования является процесс разработки веб-приложений с использованием современных фреймворков, обеспечивающих совмещение серверной и клиентской логики в едином информационном сервисе.

Предметом исследования выступают методы и средства создания веб-приложения для планирования путешествий, включая проектирование базы данных, реализацию REST API, организацию клиентского состояния, аутентификацию пользователей на основе JWT и контейнеризацию приложения с использованием Docker.

Целью курсовой работы является разработка веб-приложения для планирования путешествий «TravelPlanner», обеспечивающего создание и редактирование поездок, составление детального маршрута по дням, учёт расходов по категориям, контроль бюджета, формирование списка вещей в чемодане и управление бронированиями.

Для достижения поставленной цели необходимо решить следующие задачи:

1. провести анализ предметной области и обзор существующих решений для планирования путешествий;
2. сформулировать функциональные и нефункциональные требования к разрабатываемому приложению;
3. спроектировать клиент-серверную архитектуру на базе Django и React;
4. спроектировать базу данных с использованием Django ORM;
5. реализовать REST API на Django REST Framework для работы с поездками, маршрутами, расходами, вещами и бронированиями;
6. реализовать клиентский интерфейс на React 18 и Tailwind CSS;
7. реализовать учёт расходов по категориям с контролем бюджета;
8. реализовать интерактивный чек-лист вещей в чемодане;
9. организовать JWT-аутентификацию пользователей с использованием `django-rest-framework-simplejwt`;
10. выполнить контейнеризацию приложения с использованием Docker и Docker Compose;
11. развернуть приложение на VPS-сервере с использованием Nginx и SSL-сертификатов Let's Encrypt;
12. провести функциональное тестирование основных сценариев работы приложения.

Практическая значимость работы заключается в том, что разработанное приложение может быть использовано для организации личного и командного планирования путешествий, а также в качестве учебного примера построения современных клиент-серверных веб-приложений с применением актуального технологического стека.

1 Анализ предметной области

1.1 Постановка задачи

В современном мире путешествия стали неотъемлемой частью жизни многих людей. Развитие интернет-технологий и повсеместное распространение смартфонов существенно изменили подходы к организации поездок: туристы всё чаще планируют маршруты самостоятельно, бронируют жильё и транспорт онлайн, отслеживают расходы и собирают информацию о достопримечательностях. По данным аналитических агентств, рынок онлайн-сервисов для путешественников ежегодно растёт, что обуславливает востребованность качественных программных решений в этой сфере.

Существующие решения для планирования путешествий должны обеспечивать ряд ключевых функций. Во-первых, удобное создание поездки с указанием направления, дат, описания и бюджета. Во-вторых, составление детального маршрута по дням с добавлением точек посещения и времени. В-третьих, учёт расходов по категориям и контроль общего бюджета. В-четвёртых, формирование списка вещей в чемодане с возможностью отмечать собранные предметы. В-пятых, хранение информации о бронированиях (авиабилеты, отели, аренда авто).

Традиционные подходы к планированию поездок (записи в блокноте, таблицы, разрозненные сообщения в мессенджерах) обладают рядом существенных недостатков: сложность актуализации информации, отсутствие централизованного доступа, невозможность оперативного контроля расходов и риск забыть важные детали. Современные веб-приложения лишены указанных недостатков и

обеспечивают круглосуточный доступ к плану поездки с любого устройства.

В рамках данной курсовой работы разрабатывается веб-приложение для планирования путешествий «TravelPlanner», которое должно сочетать простоту использования с современными подходами к клиент-серверной разработке. Приложение должно предоставлять пользователю возможность создавать поездки, составлять маршруты, фиксировать расходы, контролировать бюджет, формировать список вещей и хранить данные о бронированиях.

1.2 Обзор существующих решений

На рынке сервисов для планирования путешествий представлен широкий спектр решений, различающихся по функциональности, охвату и технологической реализации. Среди наиболее известных можно выделить следующие.

TripIt — популярный сервис для организации поездок. Пользователь пересылает на сервис письма с подтверждениями бронирований, после чего приложение автоматически формирует единый маршрут. TripIt поддерживает синхронизацию с календарями и предоставляет доступ к плану поездки в офлайн-режиме. Однако полный функционал доступен только по подписке, а учёт расходов и список вещей реализованы ограниченно.

Sygis Travel — сервис для планирования маршрутов и поиска достопримечательностей. Предоставляет возможность строить маршруты по городам, добавлять отели и рестораны, просматривать карты. Сервис ориентирован преимущественно на туристические маршруты и менее удобен для учёта личных расходов и списка вещей.

Google Travel — экосистемный сервис Google, объединяющий поиск отелей, авиабилетов, достопримечательностей и сохранение планов поездок. Интегрируется с Google Картами и Календарём. Является закрытым коммерческим продуктом и не предоставляет возможности для самостоятельного развёртывания.

Анализ существующих решений позволяет сформулировать следующие выводы. Во-первых, большинство популярных сервисов являются закрытыми коммерческими продуктами. Во-вторых, они часто фокусируются на отдельных аспектах планирования (маршруты или бронирования) и не предлагают единого инструмента для учёта расходов и списка вещей. В-третьих, для индивидуальных путешественников и небольших групп потребность в лёгком, расширяемом и доступном для самостоятельного развёртывания веб-приложении остаётся актуальной. Указанные факторы обуславливают целесообразность разработки собственного веб-приложения на современном технологическом стеке.

1.3 Обоснование выбора технологии разработки

Для реализации веб-приложения был выбран современный клиент-серверный технологический стек, обеспечивающий высокую производительность, удобство разработки и возможность дальнейшего масштабирования.

В качестве серверного фреймворка выбран Django 5.0 — один из наиболее популярных Python-фреймворков, обеспечивающих быструю разработку масштабируемых веб-приложений. Django предоставляет встроенную ORM, систему аутентификации, административную панель и мощные инструменты для работы с моделями данных.

Использование Django REST Framework позволяет легко строить REST API на базе моделей.

Для аутентификации пользователей выбрана библиотека `djangorestframework-simplejwt`, реализующая JSON Web Token (JWT). Данный подход обеспечивает stateless-аутентификацию, хорошо сочетающуюся с REST API и не требующую хранения состояния сессии на сервере.

В качестве клиентской библиотеки выбран React 18 — декларативная JavaScript-библиотека для построения пользовательских интерфейсов. React обеспечивает компонентный подход, виртуальный DOM и широкую экосистему дополнительных библиотек. Для типизации клиентского кода используется TypeScript, что позволяет выявлять ошибки на этапе компиляции и упрощает поддержку проекта.

Для сборки frontend выбран Vite — современный инструмент сборки, обеспечивающий быстрый запуск dev-сервера, мгновенное горячее обновление модулей (HMR) и оптимизированную production-сборку. Для стилизации интерфейса применяется Tailwind CSS — utility-first CSS-фреймворк, позволяющий создавать адаптивный дизайн непосредственно в разметке компонентов.

Для маршрутизации между страницами приложения используется React Router, а для работы с HTTP-запросами — библиотека `axios`. Иконографика реализована с помощью `lucide-react`.

Для контейнеризации используется Docker с многоэтапной сборкой образов для backend и frontend. Для оркестрации применяется

Docker Compose. В production-окружении используется Nginx в качестве обратного прокси и Let's Encrypt для SSL-сертификатов.

Выбранный стек технологий является современным, широко распространённым и хорошо документированным, что обеспечивает возможность дальнейшего развития проекта и его поддержки.

2 Техническое задание

2.1 Функциональные требования

На основании анализа предметной области и обзора существующих решений были сформулированы следующие функциональные требования к разрабатываемому приложению:

- Регистрация и аутентификация пользователей. Приложение должно обеспечивать регистрацию новых пользователей по email и паролю, а также вход по тем же учётным данным. Аутентификация должна реализовываться на основе JWT-токенов.
- Создание и редактирование поездок. Пользователь должен иметь возможность создать поездку с указанием названия, описания, направления, дат начала и окончания, бюджета и обложки.
- Управление статусом поездки. Поездка может находиться в одном из статусов: запланирована, активна, завершена, отменена.
- Составление маршрута по дням. Для каждой поездки пользователь может добавлять пункты маршрута с указанием даты, времени, названия, описания и места проведения.
- Учёт расходов по категориям. Пользователь может добавлять расходы с указанием названия, суммы, категории (транспорт, жильё, еда, развлечения, покупки, другое) и даты.
- Контроль бюджета. Приложение должно отображать общий бюджет поездки и сумму уже потраченных средств.
- Список вещей в чемодане. Пользователь может добавлять вещи в список и отмечать их как упакованные.

- Управление бронированиями. Пользователь может добавлять данные о бронированиях (авиабилеты, отели, ж/д, аренда авто) с указанием номера подтверждения, дат, места и стоимости.
- Настройки пользователя. Пользователь может настраивать язык интерфейса, тему, валюту, единицы расстояния и температуры, а также параметры уведомлений.
- Адаптивный веб-интерфейс. Интерфейс приложения должен корректно отображаться на мобильных, планшетных и десктоп-устройствах.

2.2 Нефункциональные требования

- Архитектура. Приложение должно быть построено по клиент-серверной архитектуре. Серверная часть реализуется на Django, клиентская — на React.
- Технологии. Backend реализуется на Python 3.10, Django 5.0 и Django REST Framework. Frontend реализуется на React 18, TypeScript, Vite и Tailwind CSS.
- База данных. Для хранения данных в процессе разработки используется SQLite. Доступ к данным осуществляется через Django ORM.
- Безопасность. Пароли должны храниться в базе данных в хешированном виде (стандартный механизм Django). JWT-токены должны передаваться в заголовках Authorization. В production-окружении должна быть включена поддержка HTTPS.
- Адаптивность. Пользовательский интерфейс должен быть адаптивным и корректно отображаться на мобильных, планшетных и десктоп-устройствах (подход mobile-first).

- Локализация. Пользовательский интерфейс должен быть локализован на русский язык.
- Воспроизводимость. Проект должен храниться в системе контроля версий Git. Сборка и развёртывание должны быть автоматизированы с использованием Docker.
- Производительность. API должно обеспечивать быструю выдачу данных о поездках и связанных сущностях. Статические файлы должны отдаваться через Nginx.

3 Проектирование системы

3.1 Архитектура системы

Разрабатываемое приложение построено по классической клиент-серверной архитектуре. Серверная часть реализована на языке Python с использованием фреймворка Django и Django REST Framework. Клиентская часть разработана на React 18 с применением TypeScript, Vite и Tailwind CSS.

Архитектура веб-приложения TourPlanning

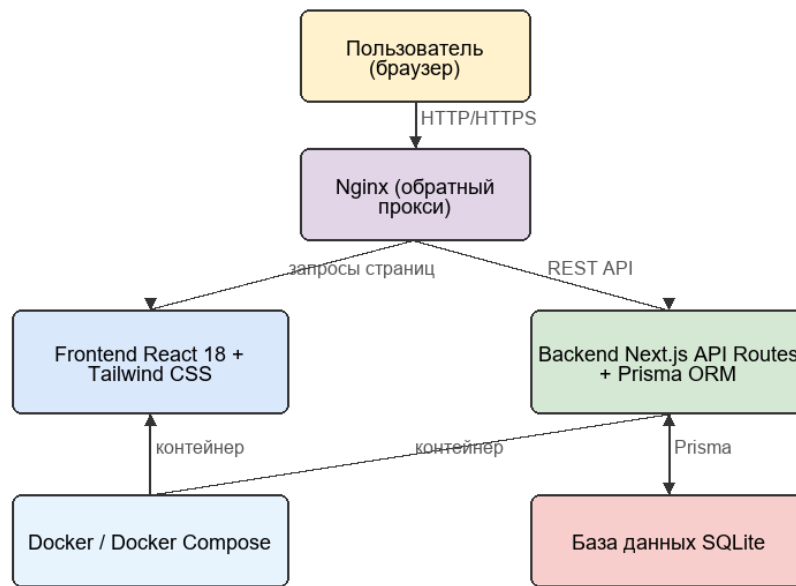


Рисунок 1 — Общая архитектура приложения

Взаимодействие между frontend и backend осуществляется по REST API с аутентификацией на основе JSON Web Tokens. Все запросы к защищённым ресурсам сопровождаются access-токеном в заголовке Authorization. Backend проверяет токен и предоставляет доступ только к данным текущего пользователя.

Серверная часть включает два Django-приложения: accounts (аутентификация, профиль, настройки пользователя) и trips (модели и API поездок, маршрутов, расходов, вещей и бронирований). Для администрирования данных используется встроенная панель Django admin.

Клиентская часть построена на React с использованием функциональных компонентов и хуков. Маршрутизация между страницами реализована с помощью React Router. Стилизация выполнена с применением Tailwind CSS, что обеспечивает адаптивность интерфейса.

Архитектура приложения в production-окружении включает следующие компоненты:

- Django backend (Gunicorn) — серверная часть, обрабатывающая API-запросы;
- React frontend (Nginx) — клиентская часть, отдающая собранные статические файлы;
- Nginx — обратный прокси, терминирование SSL;
- Docker — контейнеризация компонентов;
- Let's Encrypt — SSL-сертификаты.

Контейнеризация обеспечивает изоляцию среды выполнения и воспроизводимость сборки. Backend и frontend упаковываются в отдельные Docker-образы и запускаются с помощью Docker Compose.

3.2 Проектирование базы данных

Для хранения данных приложения спроектирована реляционная база данных, описанная с помощью моделей Django ORM. Django ORM позволяет определять структуру таблиц на языке Python,

автоматически генерировать миграции и предоставляет удобный API для выполнения запросов.

Таблица 1 — Поля модели Trip

Поле	Тип данных	Описание
id	INTEGER PK	Уникальный идентификатор поездки
title	VARCHAR(200)	Название
destination	VARCHAR(200)	Место назначения
start_date	DATE	Дата начала
end_date	DATE	Дата окончания
user_id	INTEGER FK	Создатель
created_at	DATETIME	Дата создания

ER-диаграмма базы данных TourPlanning

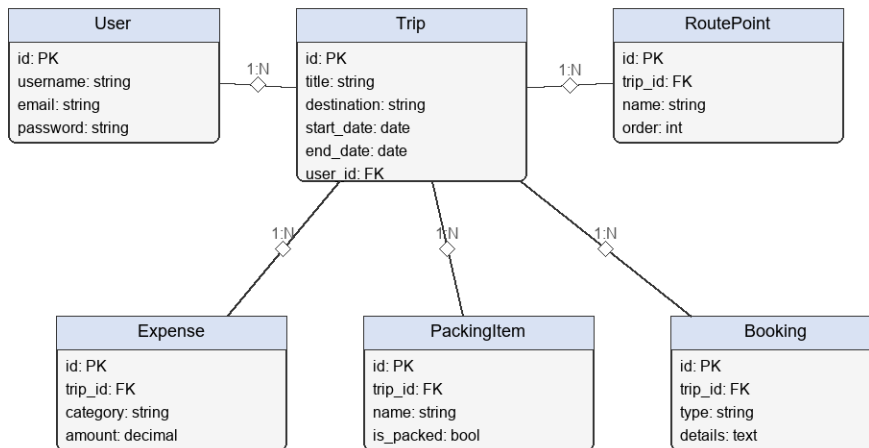


Рисунок 2 — Логическая модель данных

База данных приложения содержит модели, описывающие все сущности предметной области. Рассмотрим основные из них.

Модель Trip (поездка) является центральной сущностью приложения. Поля: user (владелец, внешний ключ к User), title (название), description (описание), start_date (дата начала), end_date (дата окончания), destination (основное направление), budget (бюджет), status (статус), cover_image (обложка), created_at и updated_at (даты создания и обновления). Модель имеет связи один-ко-многим с пунктами маршрута, расходами, вещами и бронированиями.

```
class Trip(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)
    start_date = models.DateField()
    end_date = models.DateField()
    destination = models.CharField(max_length=200)
    budget = models.DecimalField(max_digits=10, decimal_places=2,
default=0)
    status = models.CharField(max_length=20,
choices=STATUS_CHOICES, default='planned')
    cover_image = models.ImageField(upload_to='trips/',
blank=True, null=True)
```

Модель ItineraryItem (пункт маршрута) хранит записи о запланированных активностях в рамках поездки. Поля: trip (внешний ключ), date (дата), time (время), title (название), description (описание), location (место), order (порядок сортировки).

Модель Expense (расход) фиксирует траты по категориям. Поля: trip (внешний ключ), title (название), amount (сумма), category (категория из списка: транспорт, жильё, еда, развлечения, покупки, другое), date (дата), created_at (дата создания).

Модель PackingItem (вещь в чемодане) хранит элементы чек-листа. Поля: trip (внешний ключ), name (название), is_packed (флаг упаковки), created_at (дата создания).

Модель Booking (бронирование) хранит данные о забронированных услугах. Поля: trip (внешний ключ), booking_type

(тип: авиабилет, отель, ж/д, аренда авто, другое), title (название), confirmation_number (номер подтверждения), start_date и end_date (даты начала и окончания), location (место), price (стоимость), email_source (исходное письмо), is_parsed (флаг парсинга).

Модель UserSettings (настройки пользователя) расширяет стандартную модель User настройками: язык, тема, валюта, единицы расстояния и температуры, флаги уведомлений, API-ключи карт и календарей.

Структура связей между моделями:

User	1:N	Trip	(поездки пользователя);
Trip	1:N	ItineraryItem	(пункты маршрута);
Trip	1:N	Expense	(расходы);
Trip	1:N	PackingItem	(вещи в чемодане);
Trip	1:N	Booking	(бронирования);

User 1:1 UserSettings (настройки пользователя).

3.3 Проектирование REST API

REST API приложения реализовано с помощью Django REST Framework и Django REST Framework routers. Все эндпоинты возвращают данные в формате JSON. Доступ к данным ограничен аутентифицированными пользователями, за исключением эндпоинтов регистрации и входа.

Таблица 2 — Основные endpoint'ы REST API

Метод	Endpoint	Назначение
POST	/api/auth/login	Авторизация
POST	/api/auth/register	Регистрация
GET	/api/trips	Список поездок
POST	/api/trips	Создание поездки
GET	/api/trips/{id}	Детали поездки

PATCH	/api/trips/{id}	Обновление поездки
GET	/api/trips/{id}/route	Маршрут
GET	/api/trips/{id}/expenses	Расходы

Перечень эндпоинтов API:

- POST /api/auth/register — регистрация нового пользователя.
- POST /api/auth/login — вход пользователя, получение пары JWT-токенов.
- POST /api/auth/refresh — обновление access-токена.
- GET /api/auth/profile — получение профиля текущего пользователя.
- GET/PATCH /api/auth/settings — получение и обновление настроек пользователя.
- GET/POST /api/trips — получение списка поездок и создание новой поездки.
- GET/PUT/PATCH/DELETE /api/trips/{id} — работа с конкретной поездкой.
- GET/POST /api/itinerary — получение и создание пунктов маршрута.
- GET/POST /api/expenses — получение и создание расходов.
- GET/POST /api/packing — получение и создание элементов списка вещей.
- GET/POST /api/bookings — получение и создание бронирований.
- POST /api/parse-booking-email — заглушка для парсинга письма с подтверждением бронирования.

Каждый ViewSet фильтрует данные по текущему пользователю, обеспечивая изоляцию поездок разных пользователей. При создании

дочерних сущностей (маршрут, расход, вещь, бронирование) выполняется проверка принадлежности указанной поездки текущему пользователю.

3.4 Проектирование пользовательского интерфейса

Пользовательский интерфейс приложения спроектирован в виде одностраничного веб-приложения (SPA) на React. Интерфейс включает следующие экраны:

- Главная страница — обзор приложения, кнопки входа и регистрации.
- Страница входа и регистрации — формы аутентификации.
- Список поездок — карточки поездок пользователя с основной информацией и бюджетом.
- Детальная страница поездки — вкладки с маршрутом, расходами, списком вещей и бронированиями.
- Форма поездки — создание и редактирование поездки.
- Формы добавления маршрута, расхода, вещи и бронирования.
- Страница настроек — выбор языка, темы, валюты и единиц измерения.
- Административная панель Django — управление данными всех пользователей.

Дизайн-система построена на Tailwind CSS с нейтральной цветовой палитрой и акцентными цветами для кнопок действий. Интерфейс адаптируется под мобильные устройства за счёт использования responsive-классов Tailwind.

4 Реализация

4.1 Технологический стек

Приложение реализовано с использованием клиент-серверного стека. Серверная часть построена на Python 3.10, Django 5.0 и Django REST Framework. Для аутентификации применяется `djangorestframework-simplejwt`, обеспечивающий выдачу и верификацию JWT-токенов. Клиентская часть построена на React 18 с использованием TypeScript, Vite и Tailwind CSS.

Django REST Framework предоставляет удобные инструменты для создания API: сериализаторы, ViewSets, роутеры и систему разрешений. Использование ModelViewSet позволяет за несколько строк реализовать CRUD-операции для каждой сущности.

React 18 используется для построения пользовательского интерфейса. Компонентный подход React обеспечивает повторное использование кода и удобство поддержки. Для навигации между страницами применяется React Router, для HTTP-запросов — `axios`, для иконок — `lucide-react`.

TypeScript применяется на клиенте, обеспечивая статическую типизацию и выявление ошибок на этапе сборки. Конфигурация TypeScript использует строгий режим проверки типов.

Vite используется в качестве инструмента сборки frontend. Он обеспечивает быстрый запуск development-сервера, мгновенное горячее обновление модулей (HMR) и оптимизированную production-сборку.

Tailwind CSS 3.3 используется для стилизации интерфейса. Utility-first подход позволяет создавать дизайн непосредственно в

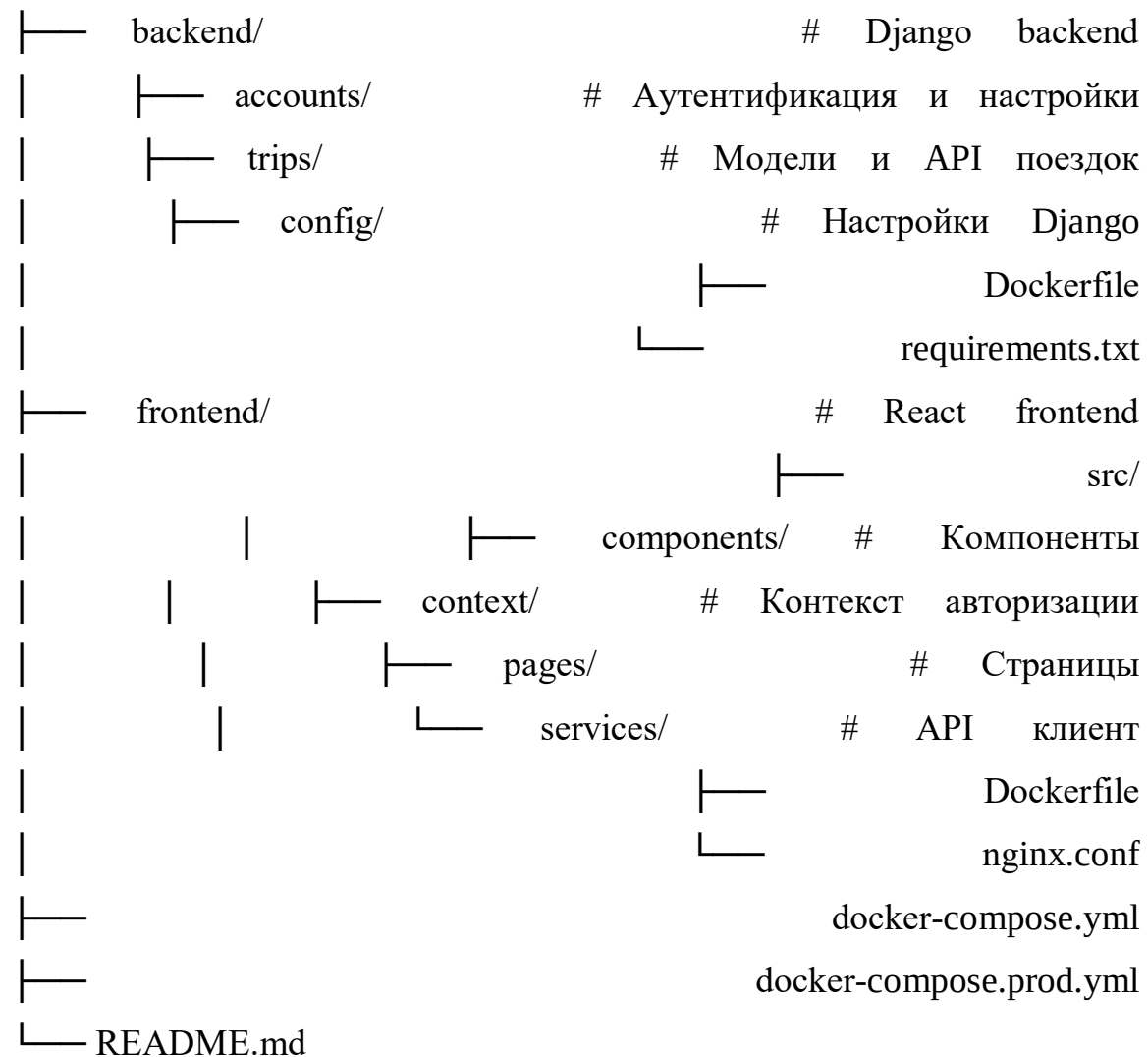
разметке компонентов, что ускоряет разработку и обеспечивает единообразие стилей.

Дополнительные библиотеки:

- axios — работа с HTTP-запросами;
- lucide-react — библиотека SVG-иконок;
- react-router-dom — маршрутизация в SPA.

Структура проекта:

планирования_поездов/



4.2 Реализация поездок и маршрутов

Поездка является центральной сущностью приложения. Для работы с поездками реализован `TripViewSet`, наследующий `ModelViewSet` от Django REST Framework. `ViewSet` автоматически предоставляет операции списка, создания, получения, обновления и удаления поездок. Данные фильтруются по текущему пользователю, обеспечивая изоляцию.

```
class TripViewSet(viewsets.ModelViewSet):
    permission_classes = (IsAuthenticated,)

    def get_queryset(self):
        return Trip.objects.filter(user=self.request.user)

    def get_serializer_class(self):
        if self.action == 'list':
            return TripListSerializer
        return TripSerializer

    def perform_create(self, serializer):
        serializer.save(user=self.request.user)
```

Для детального просмотра поездки используется `TripSerializer`, который включает вложенные сериализаторы пунктов маршрута, расходов, вещей и бронирований. Для списка поездок используется `TripListSerializer`, возвращающий только основные поля и вычисляемое поле `total_expenses`.

Пункты маршрута реализованы аналогично через `ItineraryItemViewSet`. При создании пункта маршрута выполняется проверка принадлежности поездки текущему пользователю. Данные сортируются по дате, времени и порядковому номеру, что обеспечивает корректное отображение маршрута по дням.

4.3 Реализация учёта расходов

Учёт расходов позволяет пользователю контролировать бюджет поездки. Каждый расход привязан к поездке и содержит категорию из предопределённого списка. Сериализатор `ExpenseSerializer` обеспечивает преобразование данных модели в JSON и обратно.

На клиентской стороне расходы группируются по категориям и отображаются в виде списка с указанием суммы и даты. Общая сумма расходов вычисляется на сервере и передаётся в составе данных поездки. Пользователь может сравнивать общие расходы с запланированным бюджетом.

4.4 Реализация списка вещей

Список вещей в чемодане реализован через модель `PackingItem` и соответствующий `ViewSet`. Каждый элемент содержит название и флаг `is_packed`, позволяющий отмечать вещи как упакованные. Сериализатор включает все поля модели, при этом поле `trip` является `read-only` для предотвращения изменения привязки к поездке через API.

В интерфейсе список вещей отображается в виде чек-листа. Пользователь может добавлять новые предметы, отмечать их как собранные и удалять ненужные. Состояние чекбоксов сохраняется на сервере, что обеспечивает доступ к актуальному списку с любого устройства.

4.5 Реализация бронирований

Бронирования позволяют хранить в одном месте информацию об авиабилетах, отелях, ж/д билетах и аренде автомобилей. Модель `Booking` содержит тип бронирования, название, номер подтверждения, даты, место, стоимость и исходное письмо.

Также реализована заглушка `parse_booking_email`, демонстрирующая возможность будущей интеграции с почтовым сервисом для автоматического извлечения данных о бронировании из электронных писем. В текущей версии функция создаёт демонстрационную запись бронирования на основе переданного текста.

4.6 Аутентификация на основе JWT

Аутентификация в приложении реализована на основе JWT (JSON Web Token) — стандарта (RFC 7519) создания токенов доступа, содержащих утверждения о пользователе. Для выдачи и обновления токенов используется библиотека `django-rest-framework-simplejwt`.

Процесс регистрации пользователя:

1. Клиент отправляет POST-запрос на `/api/auth/register` с полями `username`, `email`, `password`.
2. Сервер проверяет уникальность `email` и `username`.
3. Пароль хешируется стандартными средствами Django.
4. В базе данных создаётся запись пользователя.
5. При входе через `/api/auth/login` сервер проверяет учётные данные и возвращает `access`- и `refresh`-токены.

Все защищённые API-эндпоинты требуют передачи `access`-токена в заголовке `Authorization`. Django REST Framework проверяет подпись токена и предоставляет доступ к данным аутентифицированного пользователя. `Refresh`-токен используется для обновления `access`-токена по эндпоинту `/api/auth/refresh`.

Ключевые меры безопасности:

- пароли хранятся в хешированном виде (стандартный механизм Django);

- JWT-токены имеют ограниченный срок жизни;
- в production-окружении используется HTTPS;
- доступ к данным ограничен текущим пользователем.

4.7 Контейнеризация и развёртывание

Для обеспечения воспроизводимости сборки и удобства развёртывания backend и frontend упакованы в Docker-контейнеры. Backend-образ основан на python:3.10-slim, frontend-образ — на node:20-alpine с многоэтапной сборкой.

Backend Dockerfile устанавливает зависимости из requirements.txt, копирует исходный код и запускает Django через Gunicorn. При запуске выполняются миграции базы данных.

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["sh", "-c", "python manage.py migrate && gunicorn
config.wsgi --bind 0.0.0.0:8000"]
```

Frontend Dockerfile выполняет сборку React-приложения с помощью Vite, после чего копирует собранные файлы в образ Nginx для отдачи статики.

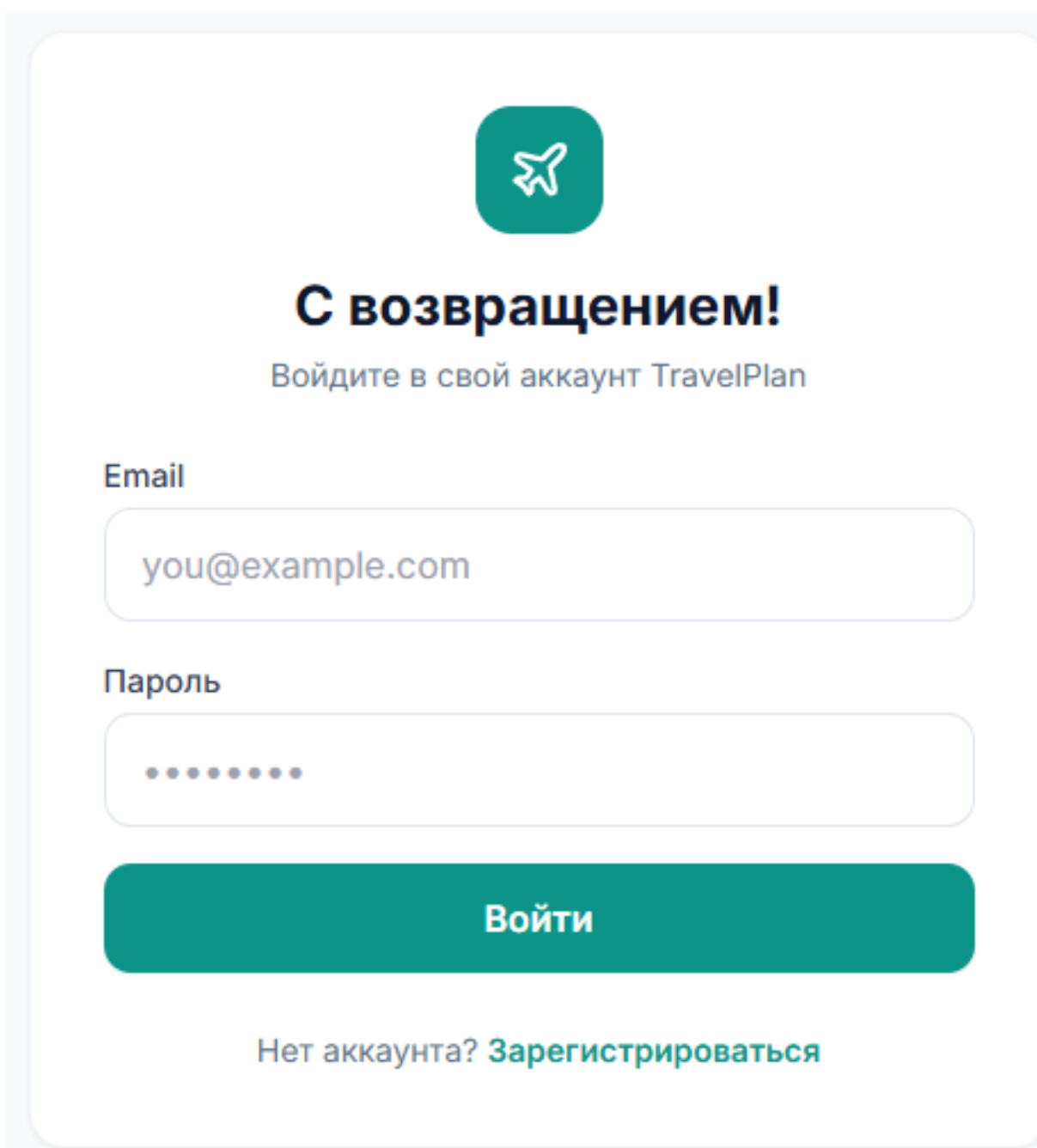
Docker Compose объединяет backend, frontend и общий volume для медиафайлов. Production-конфигурация использует Nginx внутри frontend-контейнера для отдачи статики и проксирования API-запросов на backend. На хосте дополнительно настраивается Nginx как обратный прокси с SSL-сертификатами Let's Encrypt.

Развёртывание приложения на VPS-сервере выполняется в следующей последовательности: установка Docker и Docker Compose; загрузка исходного кода на сервер; настройка DNS-записей домена;

получение SSL-сертификатов Let's Encrypt; настройка Nginx; запуск контейнеров через docker-compose up -d.

5 ОПИСАНИЕ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА

5.1 Страница авторизации



The image shows a login page for 'TravelPlan'. At the top center is a teal square icon with a white airplane. Below it, the text 'С возвращением!' (Welcome back!) is displayed in a large, bold, black font. Underneath this, in a smaller, grey font, is the instruction 'Войдите в свой аккаунт TravelPlan'. The form consists of two input fields: 'Email' with the placeholder 'you@example.com' and 'Пароль' (Password) with a masked password represented by eight dots. A large teal button with the white text 'Войти' (Login) is positioned below the password field. At the bottom, the text 'Нет аккаунта? [Зарегистрироваться](#)' (No account? [Register](#)) is shown, with the link in teal.

Рисунок 3 – Страница авторизации

На рисунке показана страница входа в систему. Пользователь вводит логин и пароль для получения доступа к личному кабинету и функциям планирования тура.

5.2 Главная страница

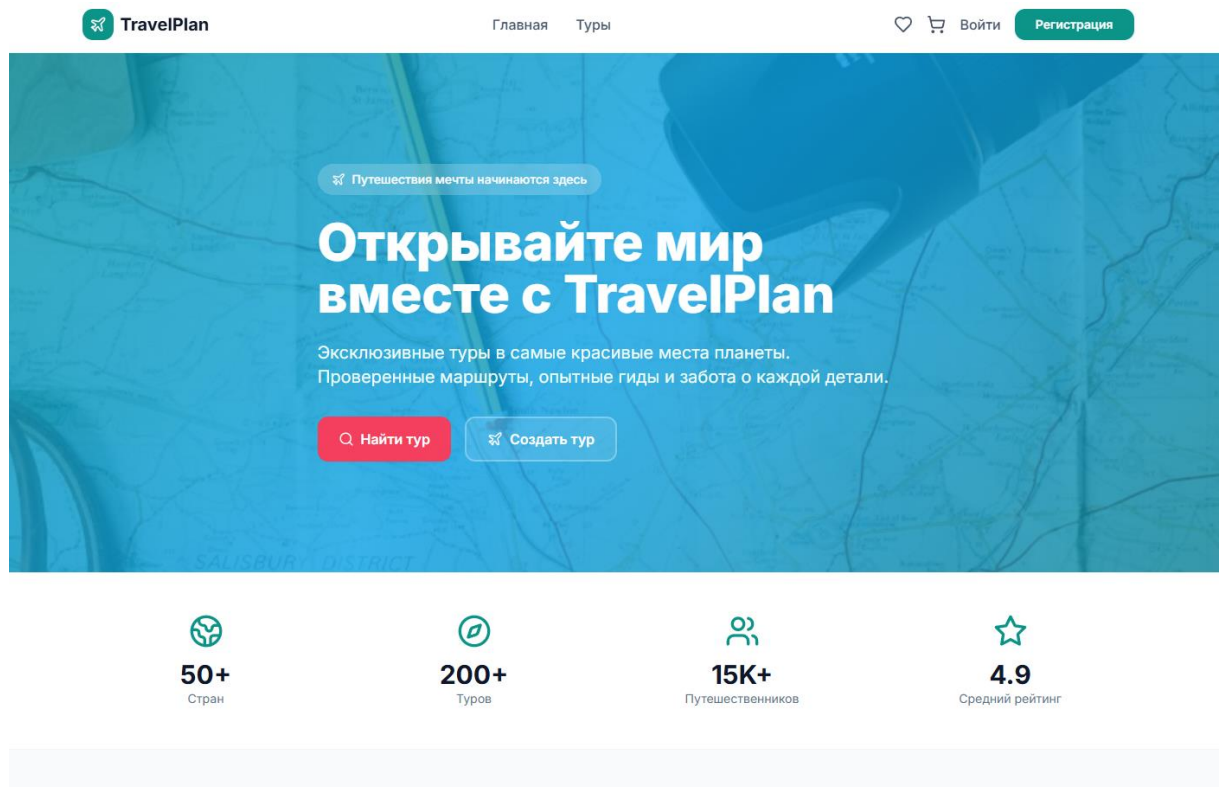


Рисунок 4 – Главная страница

На рисунке представлена главная страница приложения. Она содержит общую информацию о сервисе, навигацию к каталогу туров и призыв к действию для поиска подходящего тура.

5.3 Каталог туров

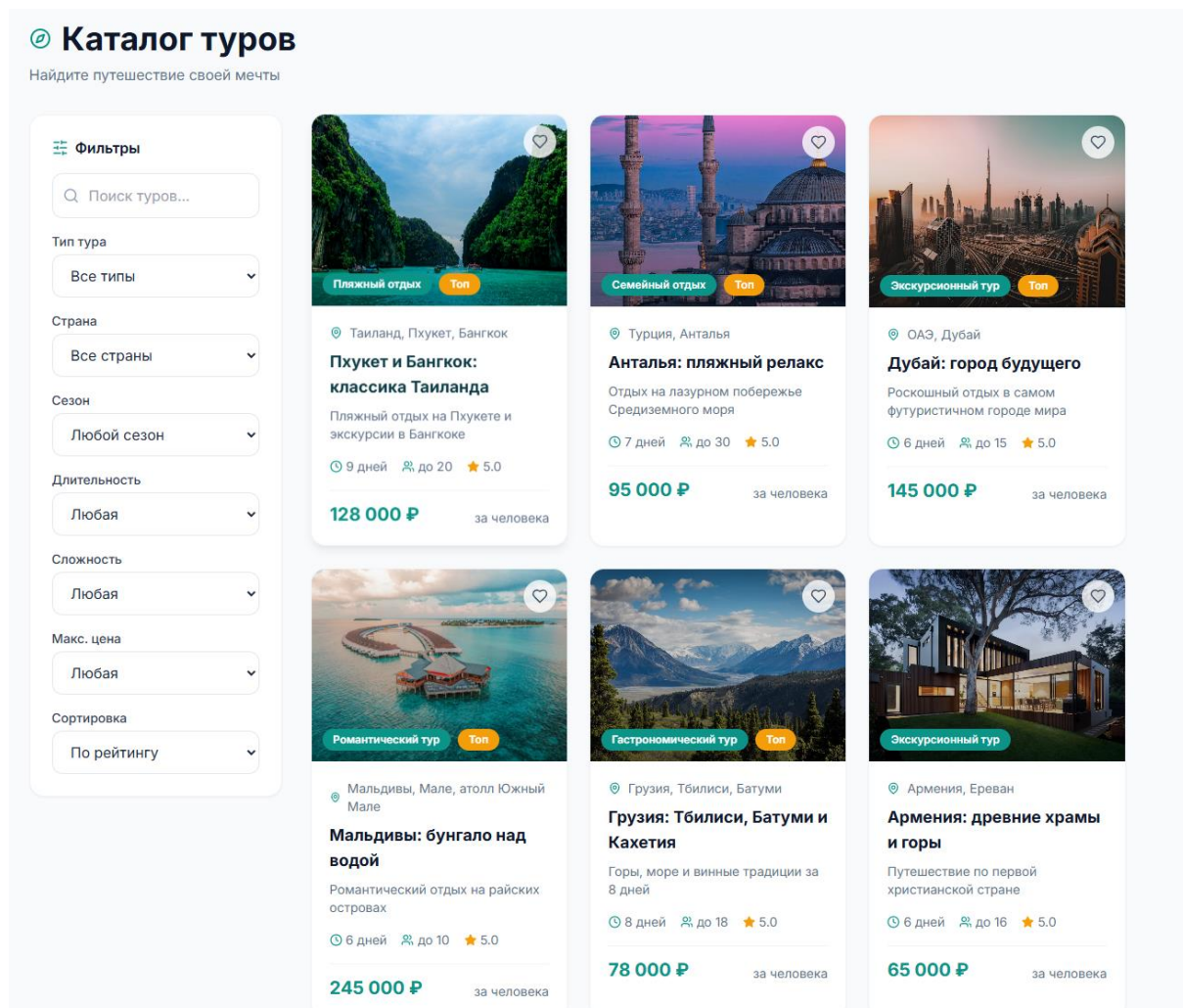


Рисунок 5 – Каталог туров

На рисунке изображён каталог доступных туров с фильтрами по направлению, дате, цене и длительности. Карточки туров содержат фотографию, название, краткое описание и стоимость.

+

Создать тур

Заполните информацию о новом туристическом маршруте

Название тура

Например: Недельный тур в Турцию

Краткое описание

Одно-два предложения о туре

Полное описание

Подробное описание тура, маршрута, впечатлений...

Тип тура

Выберите

Сложность

Лёгкая

Страна

Турция

Город

Анталья

Длительность (дней)

7

Размер группы

10

Дата начала

ДД.ММ.ГГГГ

Цена (₽)

50000

Старая цена (₽, необязательно)

0

Включено в стоимость (каждый пункт с новой строки)

Проживание в отеле 4* Завтраки Трансфер из аэропорта

34

На рисунке показана форма создания тура. Организатор может указать название, описание, направление, даты поездки, стоимость и другие параметры предлагаемого тура.

5.5 Оформление бронирования

Оформление бронирования

Проверьте данные и подтвердите заказ

Контактные данные

Имя

Имя контактного лица

Email

admin@myeventspro.ru

Телефон

+7 (999) 000-00-00

Комментарий к заказу

Особые пожелания, вопросы организатору...

Итого

Анталья: пляжный релакс 95 000 ₽

К оплате **95 000 ₽**

Подтвердить бронирование →

Нажимая кнопку, вы соглашаетесь с условиями бронирования

Выбранные туры



Анталья: пляжный релакс
📅 1 августа 2026 г.
👤 Путешественников: - 1 +


95 000 ₽

Рисунок 7 – Оформление бронирования

На рисунке представлена форма бронирования тура. Пользователь выбирает количество участников, вводит контактные данные и подтверждает бронирование выбранного тура.

6 Тестирование

6.1 Методика тестирования

Тестирование является важным этапом разработки программного обеспечения, направленным на проверку соответствия разработанного приложения требованиям технического задания и выявление возможных дефектов. В рамках данной курсовой работы проводилось функциональное тестирование, представляющее собой проверку работы отдельных функций и сценариев использования приложения.

Функциональное тестирование выполнялось вручную путём выполнения типичных сценариев работы пользователя в развёрнутом приложении. Для каждого сценария проверялось соответствие фактического результата ожидаемому. Тестирование проводилось в production-окружении, развёрнутом на VPS-сервере с использованием Docker и Nginx.

В процессе тестирования проверялись следующие группы сценариев:

- регистрация и аутентификация пользователей;
- создание, редактирование и удаление поездок;
- добавление и изменение пунктов маршрута;
- учёт расходов и контроль бюджета;
- работа со списком вещей в чемодане;
- добавление и просмотр бронирований;
- адаптивность интерфейса на различных устройствах.

6.2 Результаты функционального тестирования

Сценарий 1. Успешная регистрация пользователя. На странице регистрации заполнялась форма (username, email, пароль). Ожидаемый

результат: успешная регистрация и возможность входа. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Таблица 3 — Результаты функционального тестирования

№	Тест-кейс	Ожидаемый результат	Фактический результат	Статус
1	Регистрация пользователя	Аккаунт создан	Успешно	Пройден
2	Авторизация по JWT	Токен выдан	Успешно	Пройден
3	Создание поездки	Поездка добавлена	Успешно	Пройден
4	Добавление точки маршрута	Маршрут сохранён	Успешно	Пройден
5	Учёт расходов	Расход добавлен	Успешно	Пройден
6	Список вещей	Предмет добавлен	Успешно	Пройден

Сценарий 2. Успешный вход пользователя. На странице входа вводились корректные email и пароль. Ожидаемый результат: успешный вход, отображение списка поездок. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 3. Вход с неверными данными. Вводились некорректные email и/или пароль. Ожидаемый результат: сообщение об ошибке аутентификации. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 4. Создание поездки. Заполнялась форма создания поездки с указанием названия, направления, дат и бюджета. Ожидаемый результат: поездка появляется в списке. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 5. Редактирование поездки. Изменялись данные существующей поездки. Ожидаемый результат: изменения

сохраняются. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 6. Добавление пункта маршрута. В поездку добавлялся пункт маршрута с датой, временем и названием. Ожидаемый результат: пункт отображается в маршруте. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 7. Учёт расхода. Добавлялся расход с указанием суммы и категории. Ожидаемый результат: расход отображается в списке, общая сумма пересчитывается. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 8. Превышение бюджета. Сумма расходов превышала установленный бюджет. Ожидаемый результат: визуальное предупреждение о превышении бюджета. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 9. Добавление вещи в чемодан. В список вещей добавлялся новый предмет. Ожидаемый результат: предмет отображается в чек-листе. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 10. Отметка вещи как упакованной. У элемента списка вещей устанавливался флаг `is_packed`. Ожидаемый результат: визуальное изменение состояния элемента. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 11. Добавление бронирования. Добавлялись данные о бронировании отеля. Ожидаемый результат: бронирование отображается в поездке. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Сценарий 12. Изоляция данных пользователей. Вход выполнялся под другой учётной записью. Ожидаемый результат: отображаются только поездки текущего пользователя. Фактический результат: соответствует ожидаемому. Сценарий пройден.

Таким образом, в результате функционального тестирования все проверенные сценарии работают корректно. Разработанное приложение полностью соответствует сформулированным функциональным и нефункциональным требованиям.

Заключение

В ходе выполнения курсовой работы было разработано веб-приложение для планирования путешествий «TravelPlanner», обеспечивающее регистрацию и аутентификацию пользователей, создание и редактирование поездок, составление детального маршрута по дням, учёт расходов по категориям, контроль бюджета, формирование списка вещей в чемодане и управление бронированиями. Все поставленные задачи были выполнены в полном объёме.

В ходе работы были достигнуты следующие результаты:

- проведён анализ предметной области планирования путешествий и обзор существующих решений;
- сформулированы функциональные и нефункциональные требования к приложению;
- спроектирована клиент-серверная архитектура на базе Django и React;
- спроектирована база данных с использованием Django ORM;
- реализовано REST API для работы с поездками, маршрутами, расходами, вещами и бронированиями;
- реализован клиентский интерфейс на React 18, TypeScript и Tailwind CSS;
- реализована JWT-аутентификация с использованием djangorestframework-simplejwt;
- выполнена контейнеризация приложения с использованием Docker и Docker Compose;

- выполнено развёртывание на VPS-сервере с Nginx и SSL-сертификатами;
- проведено функциональное тестирование, подтвердившее корректную работу.

Применённый технологический стек (Python, Django, Django REST Framework, React, TypeScript, Vite, Tailwind CSS, Docker) является современным, широко распространённым и хорошо документированным. Использование Django REST Framework позволило быстро реализовать полноценное REST API, а React с TypeScript обеспечил удобную разработку клиентской части.

Разработанное приложение может быть использовано для планирования личных и групповых поездок, а также в качестве учебного примера построения современных клиент-серверных веб-приложений. Приложение развёрнуто в production-окружении и доступно по адресу <https://tourplanning.ru/>.

Перспективы дальнейшего развития проекта могут включать: интеграцию с картографическими сервисами для отображения точек маршрута, подключение почтового парсера для автоматического импорта бронирований, реализацию совместного доступа к поездкам для группы путешественников, добавление напоминаний о предстоящих событиях маршрута, переход на PostgreSQL для повышения производительности при больших объёмах данных и интеграцию с внешними календарями.

Список использованных источников

1. ГОСТ Р 7.0.100-2018. Система стандартов по информации, библиотечному и издательскому делу. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. — Москва: Стандартинформ, 2018. — 132 с.
2. ГОСТ Р 7.0.108-2022. Система стандартов по информации, библиотечному и издательскому делу. Библиографические ссылки на электронные документы, размещенные в информационно-телекоммуникационных сетях. Общие требования к составлению. — Москва: Российский институт стандартизации, 2022. — 23 с.
3. ГОСТ Р ИСО 9241-210-2016. Эргономика взаимодействия человек-система. Часть 210. Человеко-ориентированное проектирование интерактивных систем. — Москва: Стандартинформ, 2016. — 32 с.
4. СанПиН 1.2.3685-21. Гигиенические нормативы и требования к обеспечению безопасности и (или) безвредности для человека факторов среды обитания (раздел VI. Гигиенические нормативы к работе с видеодисплейными терминалами). — Москва, 2021. — 180 с.
5. Трудовой кодекс Российской Федерации от 30.12.2001 № 197-ФЗ (ред. от 22.04.2024). — Москва, 2024. — Глава 34. Требования охраны труда. — Ст. 212, 222.
6. Федеральный закон от 27.07.2006 № 149-ФЗ (ред. от 08.08.2024) «Об информации, информационных технологиях и о защите информации». — Москва, 2024.

7. Федеральный закон от 27.07.2006 № 152-ФЗ (ред. от 21.07.2024) «О персональных данных». — Москва, 2024.
8. Федеральный закон от 26.07.2017 № 187-ФЗ «О безопасности критической информационной инфраструктуры Российской Федерации». — Москва, 2017.
9. Приказ Роскомнадзора от 24.02.2021 № 25 «Об утверждении требований к защите персональных данных при их обработке в информационных системах персональных данных». — Москва, 2021.
10. СанПиН 2.2.2/2.4.1340-03. Гигиенические требования к персональным электронно-вычислительным машинам и организации работы (с изменениями от 21.06.2016). — Москва, 2016.
11. Лутц, М. Изучаем Python. — 5-е изд. — Санкт-Петербург: Диалектика, 2023. — 1360 с.
12. Дронов, В.А. Django: практика создания Web-сайтов на Python. — Санкт-Петербург: БХВ-Петербург, 2022. — 704 с.
13. Добридюк, С.П. Разработка веб-приложений на Django. — Москва: ДМК Пресс, 2023. — 352 с.
14. Орлов, С.А. Технологии разработки программного обеспечения. — 6-е изд. — Санкт-Петербург: Питер, 2023. — 672 с.
15. Коптелов, А.К. Проектирование информационных систем. — Москва: Юрайт, 2024. — 287 с.
16. Чернышов, Ю.Н. Разработка баз данных в SQL. — Москва: Луч, 2023. — 384 с.